ENGINEERING HANDOVER

Spring QC

Codebase handover document — architecture, data model, API surface, configuration, known issues, and gotchas, derived directly from the repository.

Generated for onboarding a new developer • spring-qc-app + spring-qc-exe

Prepared as an onboarding reference for a new developer. Everything below is based on what's actually in the repository as of this document's creation — anywhere the code didn't answer a question, it says so instead of guessing.

---

1. Project Overview

Spring QC is a vision-based quality-control station for a spring assembly line. A YOLO model detects and tracks springs through a camera's region of interest (ROI); when a spring leaves the ROI, a human operator marks it **Not defective**, **Defective**, or **Skip** from captured photos, and every decision — with timing and the captured frames — is stored in PostgreSQL for reporting. It's built for a single shop-floor deployment: one "edge box" (a PC with an NVIDIA GPU) runs the whole stack, and the admin and every operator touch-panel just point a browser at it over the local network — nothing installs on those other machines.

Repo structure

```
D:\SPRING\
├── spring-qc-app\      # THE PRODUCT — fully self-contained, independent app.
│                        # Nothing outside this folder is required to run it.
│   ├── app.py           # FastAPI server: pages, operators/assignments/config
│   │                      APIs, job manager, reports, Excel export, websocket
│   ├── detector.py       # YOLO + ByteTrack detection/tracking core
│   ├── config.yaml       # Model thresholds, ROI default, tracker tuning
│   ├── model/best.pt     # Trained YOLO weights (~19 MB)
│   ├── trackers/         # Tuned ByteTrack config
│   ├── static/           # 5 server-rendered-by-FastAPI HTML pages + shared
│   │                      nav.js / app.css (vanilla JS, no frontend build step)
│   ├── deploy/            # On-site deployment docs + kiosk-mode scripts
│   ├── Dockerfile, docker-compose.yml
│   ├── migrate_from_sqlite.py   # one-shot legacy data import
│   └── requirements.txt, run.bat, setup.bat
│
├── spring-qc-exe\       # Packaging/deployment layer that wraps spring-qc-app
│                        # as native Windows executables. References
│                        # spring-qc-app's source at build time; never forks it.
│   ├── server\            # SpringQCServer.exe / SpringQCService.exe — the
│   │                      GPU inference box, console or Windows-Service form
│   ├── desktop\           # SpringQCDesktop.exe — same server, windowed UI
│   │                      (no console box) + system tray, replaces server\
│   │                      for day-to-day use
│   ├── hmi\               # SpringQCHMI.exe — optional locked-down kiosk
│   │                      browser shell for a touch panel
│   ├── client-shortcuts\  # Zero-install browser shortcuts (Windows .url /
│   │                      Linux .desktop) for every other LAN PC — this is
│   │                      what "HMI" means day to day: a plain browser
│   ├── client-report\      # A generated client-facing PDF/HTML product overview
│   └── GUIDE.md, README.md
│
├── venv\                # Old, unused top-level dev virtualenv (predates
│                         # the spring-qc-app/spring-qc-exe split)
├── Videos\               # Sample/reference footage, not part of the app
├── trash\                # Superseded pre-refactor prototype files, moved
│                         # aside rather than deleted (see §9)
├── .claude\               # AI-assistant tooling config, not app-related
└── .git\                  # Repo root (see §10 — currently has zero commits)
```

Key fact for a new developer: `spring-qc-app\` is the entire product and is self-contained — "no paths point outside it" (the project's own README's words). `spring-qc-exe\` only ever reads from `spring-qc-app\` at build time; it never modifies or forks the application code. If you're fixing a bug or adding a feature, you almost always want `spring-qc-app\`.

---

2. Tech Stack

Backend

Language	Python (Dockerfile pins **3.12-slim**; README says 3.10+ for non-Docker)
Web framework	**FastAPI** ≥0.110.0, served by **uvicorn[standard]** ≥0.29.0
Validation	**Pydantic** ≥2.0.0
ML / CV	**ultralytics** (YOLO) ≥8.2.0, **torch** ≥2.0.0, **torchvision** ≥0.15.0, **opencv-python-headless** ≥4.8.0, **numpy** ≥1.24.0
Database driver	**psycopg[binary]** ≥3.1 (psycopg3, not psycopg2)
Reporting export	**openpyxl** ≥3.1, **pillow** ≥10.0.0
Uploads	**python-multipart** ≥0.0.9
Config	**PyYAML** ≥6.0

Exact pins are floor-only (`>=`); there is no lockfile, so two installs at different times can resolve different exact versions. Worth flagging in §9.

Database / storage

- **PostgreSQL 16** (via the `postgres:16` Docker image) — the only database; no cache, no message queue.
- Captured defect images and exported files live on **local disk** (`Output/defects/...` , `uploads/`), not in the database or object storage.
- An old **SQLite** `qc.db` file sits in the repo root as a leftover — `migrate_from_sqlite.py` exists specifically to one-time import it into Postgres; SQLite is not used at runtime by the current app.

Frontend

- Plain HTML + vanilla JavaScript, no framework, no build step, no bundler. Five pages served directly by FastAPI from `static/` : `index.html` , `admin.html` , `operator.html` , `monitor.html` , `reports.html` , sharing `nav.js` (navbar + theme toggle + tooltips) and `app.css` .
- Live video preview and event stream go over a raw WebSocket (`/ws/{job_id}`) — JPEG frames + JSON events interleaved on one socket.

Third-party services / APIs

None — Not determinable evidence of any external API integration (no cloud ML service, no payment/email/SMS provider, no analytics). The only network dependency at runtime is the operator's own RTSP camera stream.

Build tools / package manager / runtime

- `pip` + `requirements.txt` (no Poetry/pipenv/uv).
- **Docker** + **Docker Compose** is the recommended run path.
- Native Windows packaging (separate from the app itself) uses **PyInstaller** ≥6.10 — see `spring-qc-exe\` .
- No JS package manager — there is no `package.json` anywhere in the repo.

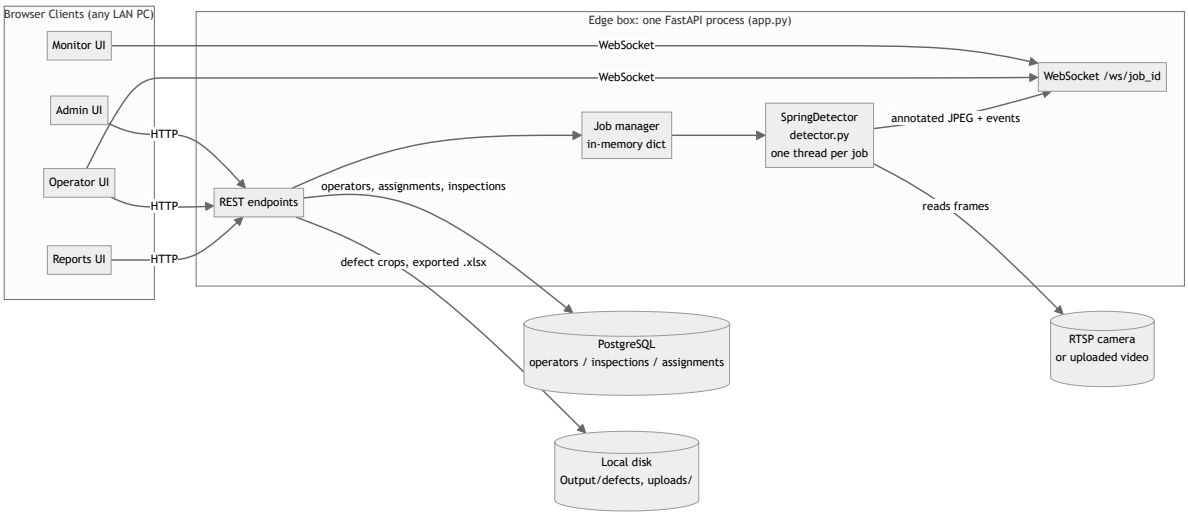
---

3. Architecture

Components and how they interact

One Python process (`app.py` , run by uvicorn) is the entire backend: it serves the five HTML pages, exposes a REST + WebSocket API, and owns an in-memory `jobs` dict mapping `job_id → Job` . Each `Job` wraps one running `SpringDetector` (from `detector.py`) in its own background thread, reading frames from either an RTSP stream or an

uploaded video file. There is no separate worker process, task queue, or message broker — concurrency is plain Python `threading`, guarded by a couple of `threading.Lock`s (`jobs_lock`, per-job `frame_lock` / `events_log_lock`).



Request/data flow — the main path (one spring's life cycle)

1. **Admin** assigns a camera/video + draws an ROI for an operator (`POST /assignments/{operator}`, `POST /operators/{name}/roi`), then flips the assignment **Live**, which calls `POST /start`.
2. `app.py` spins up a `Job` on its own thread; `Job.run()` constructs a `SpringDetector` and calls its `.run()` loop, which pulls frames via OpenCV, runs YOLO detection + ByteTrack tracking per frame, and checks each tracked box's center against the ROI polygon.
3. A track must sit inside the ROI for `confirm_frames` **consecutive** frames before it's confirmed as a real spring entry (`enter` event, a new `s_no` serial assigned) — this filters one-off misdetections. It's declared `exit` after `out_frames` consecutive frames **not** seen inside the ROI (tolerates brief occlusion without false-triggering an exit). ID loss during occlusion is bridged by `_recover_id` — a new/untracked detection within `recover_dist` pixels of a track lost within `recover_frames` frames is treated as the same spring.
4. On `exit`, `detector.py` finalizes a "pending" record (bbox crops + timing) and pushes an `exit` event over the job's broadcast log; `app.py` relays it to any connected WebSocket (Operator UI, Monitor UI).
5. **Operator** sees the captured frames and taps a verdict, which calls `POST /save-defect`; this writes the crop images to `Output/defects/<job_id>/<s_no>/` and inserts one row into `inspections`.
6. **Reports** UI queries `/reports/*` endpoints for aggregates, or exports a `.xlsx` via `/reports/export.xlsx` with thumbnails embedded per row.

Authentication and authorization

There is none, at the HTTP/API layer. Every endpoint — including `/admin`, every operator-management call, and the inference start/stop calls — is open to anyone who can reach the port. Operator "sign-in" (`POST /operators/verify`) checks a salted SHA-256 PIN hash against the database, but on success the client just stores the operator's name in **browser** `localStorage` (`static/operator.html`) — there is no server session, token, or cookie, and no API call is actually scoped to "who is signed in." This is explicitly called out as a known limitation in `spring-qc-app/deploy/README.md`: the deployment model is "keep the edge box off any network with internet egress," not application-level auth. See §9 and §11 for the practical implications.

---

4. Features

Admin — operator & camera management

- **What**: add/edit operators (name, machine number, 4-digit PIN), assign an RTSP camera or an uploaded test video, draw/edit each operator's own ROI polygon, toggle an assignment **Live**.
- **Files**: `app.py` (`/operators*`, `/assignments*`, `/start`, `/stop`, `/frame`, `/upload`), `static/admin.html`.

- **Endpoints**: `GET/POST /operators`, `POST /operators/{name}/roi`, `DELETE /operators/{name}`, `GET/PUT/DELETE /assignments/{operator}`, `GET /frame`, `POST /upload`, `POST /start`, `POST /stop/{job_id}`, `GET /jobs`.
- **Non-obvious logic**: each operator has their **own** ROI and their **own** continuously-incrementing S.No sequence (queried from `MAX(s_no)` per operator on each `/start`, not reset per job). An operator can only have one running job at a time (`409` if already assigned). Saving a new ROI pushes it live to any already-running job for that operator on its very next frame (`det.set_roi`), no restart needed.

Admin — inference configuration

- **What**: tune detection confidence/IoU thresholds, tracker choice, ROI timing knobs, from a UI instead of hand-editing YAML.
- **Files**: `app.py` (`/config`, `/save-config`, `/save-roi`, `/gpu`), `config.yaml`, `static/admin.html` Settings panel.
- **Non-obvious logic**: `/save-config` rewrites `config.yaml` on disk (not just an in-memory value) — changes apply to the **next** `/start` call, not to an already-running job. `/gpu` reports whether CUDA is actually available on this process right now, used to warn the admin if a GPU build somehow ended up running on CPU.

Operator decision station

- **What**: PIN sign-in, then a full-screen "waiting for next spring" view; when a spring exits the ROI, shows its captured photos and three buttons: Not Defective / Defective / Skip.
- **Files**: `static/operator.html`, `app.py` (`/operators/verify`, `/save-defect`, `/defect-image/{job_id}/{s_no}/{frame_no}`, the `/ws/{job_id}` websocket).
- **Non-obvious logic**: "Skip" and "unknown" are the same thing server-side — any spring never explicitly decided (operator closes the tab, job stops) is auto-recorded as `defect='unknown'` by `flush_pending_unknown()` when the job ends, so no spring silently vanishes from the count.

Live Monitor

- **What**: read-only live view of any currently-running operator's camera feed + event timeline, for a supervisor.
- **Files**: `static/monitor.html`, same `/ws/{job_id}` websocket, `/jobs`.
- **Non-obvious logic**: purely a second WebSocket viewer of the same job — it does not and cannot submit a defect decision; only the operator's own `/save-defect` call does that.

Reports & Excel export

- **What**: day-wise and per-operator dashboards (totals, defect rate, trend chart), a searchable recent-inspections table with photos, and an `.xlsx` export with thumbnails embedded.
- **Files**: `app.py` (`/reports/summary`, `/reports/operators-summary`, `/reports/daily`, `/reports/records`, `/reports/export.xlsx`), `static/reports.html`.
- **Non-obvious logic**: IN/OUT "clock" times shown in the export are reconstructed as `started_at + offset_seconds`, with a fallback to `decided_at - duration` for rows saved before `started_at` existed in the schema (an in-place migration, see §5). The export caps embedded images at 1500 total and silently stops embedding (but keeps listing rows) past that, noting the truncation on the Summary sheet.

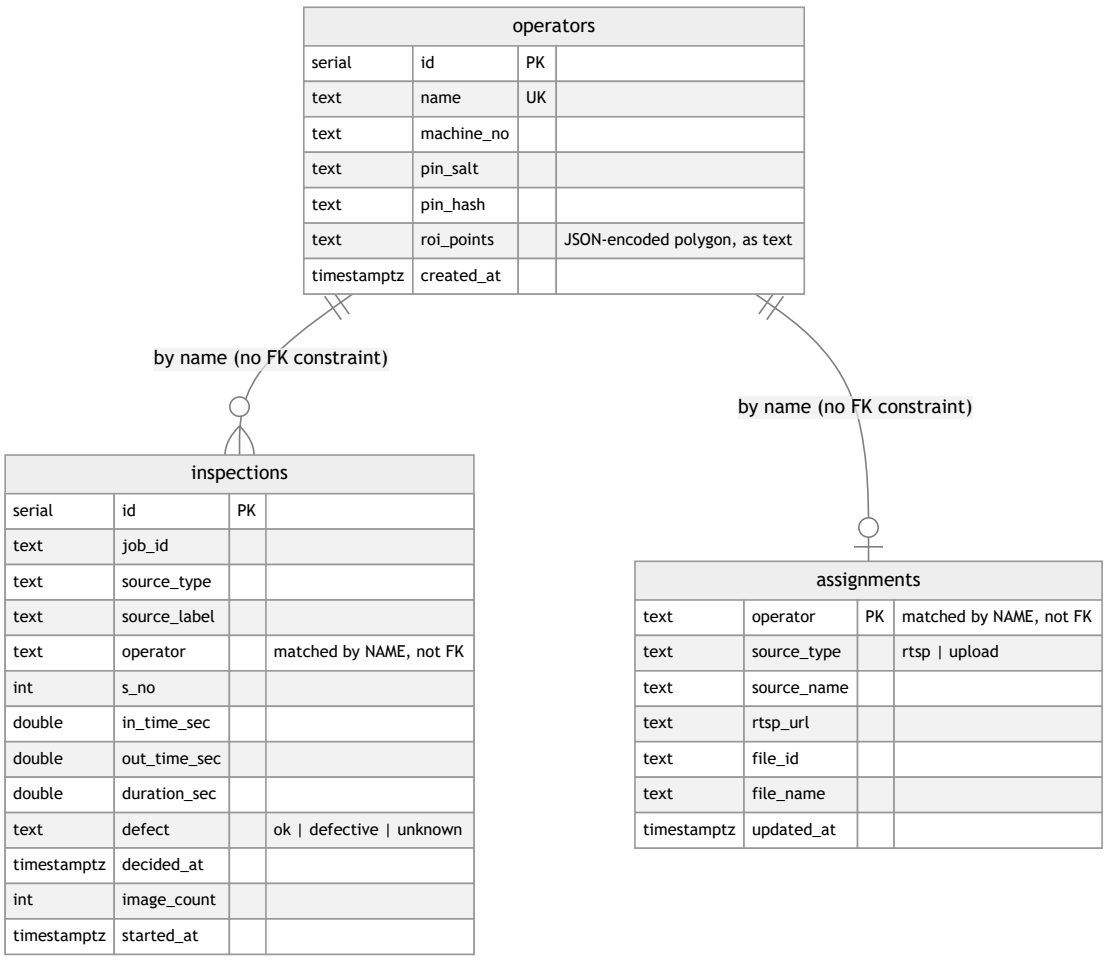
Background jobs / cron / webhooks

None found. There is no scheduler, no cron-style periodic task, and no outbound webhook anywhere in the codebase — every long-running piece of work is a single detection `Job` thread started by an explicit `/start` call and ended by `/stop` or the source running out.

---

5. Data Model

One PostgreSQL database (`spring_qc`), three tables, created by `init_db()` in `app.py` on every process start (idempotent `CREATE TABLE IF NOT EXISTS` + `ADD COLUMN IF NOT EXISTS` — see §5 migration approach below). **No ORM** — all access is hand-written SQL via `psycopg`.



Important: `operators.name` is the join key everywhere, but there are **no actual foreign-key constraints** — `inspections.operator` and `assignments.operator` are just `TEXT` columns matched by value. Deleting an operator (`DELETE /operators/{name}`) removes their `assignments` row but **leaves their `inspections` history in place with a now-dangling name**. See §9/§11.

Migration approach

There is no migration framework (no Alembic, no raw numbered SQL files). Schema evolution is handled inline in `init_db()` using `CREATE TABLE IF NOT EXISTS` plus repeated `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` statements (e.g. `started_at` was added to `inspections` this way, with the export code explicitly handling rows that predate it). This means the schema is defined in exactly one place — the top of `app.py` — and runs on every single app start, against whatever database `DATABASE_URL` points to.

`migrate_from_sqlite.py` is a separate, manually-run one-shot script for importing a legacy SQLite `qc.db` into Postgres; it is not part of normal startup and is safe to re-run (skips already-present rows).

---

6. API Surface

All paths are relative to the app root (default `http://<host>:8001`). **Auth column reflects reality, not intent** — see §3.

Method	Path	Purpose	Auth
GET	`/`	Station picker page	None
GET	`/admin`	Admin UI	None
GET	`/operator`	Operator UI	None

Method	Path	Purpose	Auth
GET	`/monitor`	Monitor UI	None
GET	`/reports`	Reports UI	None
GET	`/operators`	List operators (+ live assignment status)	None
POST	`/operators`	Create/update an operator (name, machine no, PIN)	None
POST	`/operators/{name}/roi`	Save that operator's ROI polygon	None
POST	`/operators/verify`	Check a PIN (client stores result in `localStorage`, no server session)	None
DELETE	`/operators/{name}`	Delete operator (blocked if they have a running job)	None
GET	`/assignments`	List all saved operator→source assignments	None
PUT	`/assignments/{operator}`	Create/update one assignment (RTSP or uploaded video)	None
DELETE	`/assignments/{operator}`	Remove an assignment	None
GET	`/config`	Current `config.yaml` values	None
POST	`/save-config`	Update model/tracking/events config (writes `config.yaml`)	None
GET	`/gpu`	Whether CUDA is available + which device inference runs on	None
POST	`/save-roi`	Save the global default ROI (`config.yaml`)	None
GET	`/frame`	Grab one still frame from an RTSP URL or uploaded file (for ROI drawing)	None
POST	`/upload`	Upload a test video (streamed to disk, up to 4 GiB)	None
POST	`/start`	Start a detection job against an RTSP/upload source	None
POST	`/stop/{job_id}`	Signal a running job to stop	None
GET	`/jobs`	List jobs (optionally filtered by operator)	None
POST	`/save-defect`	Record an operator's verdict for one spring	None
GET	`/defect-image/{job_id}/{s_no}/{frame_no}`	Fetch one captured defect-crop JPEG	None
GET	`/reports/summary`	Totals (ok/defective/unknown), filterable by operator/date/days	None
GET	`/reports/operators-summary`	Per-operator totals	None
GET	`/reports/daily`	Per-day counts for trend charts	None
GET	`/reports/records`	Recent inspection rows (table/list)	None
GET	`/reports/export.xlsx`	Download an Excel export with embedded images	None
WS	`/ws/{job_id}`	Live annotated-frame JPEGs + event stream for one job	None
GET	`/static/*`	Static assets (served by `StaticFiles` mount)	None

(Request/response body shapes are defined as Pydantic models directly in `app.py` — `StartRequest`, `OperatorCreate`, `OperatorVerify`, `OperatorRoiUpdate`, `AssignmentUpsert`, `ConfigUpdate`, `RoiUpdate`, `DefectUpdate` — read those for exact field names rather than duplicating them here.)

7. Configuration and Environment

Environment variables

Name	Purpose	Required?
`DATABASE_URL`	PostgreSQL connection string (`postgresql://user:pass@host:port/db`)	No — defaults to `postgresql://spring_qc:spring_qc@localhost:5432/spring` in `app.py` , i.e. a weak default credential (see §9)
`POSTGRES_PASSWORD`	Sets the `db` container's Postgres password, and is substituted into `app` 's `DATABASE_URL` , via `docker-compose.yml`	No — defaults to `spring_qc` if `.env` isn't set up; the README explicitly instructs changing it
`TORCH_INDEX`	Which PyTorch wheel index to build against (GPU `cu124` vs CPU) — a Docker **build arg**, not a runtime env var	No — defaults to the CUDA 12.4 index

Config file

`config.yaml` (not an env var, but the other half of runtime configuration) — model path/thresholds/device, default ROI points, and tracking/event tuning. Editable directly or via the Admin Settings panel (`POST /save-config` , `POST /save-roi`), which rewrites this same file.

Local vs. staging vs. production

Not determinable from code — there is no staging/production environment distinction anywhere in the repo (no environment-specific config files, no deployment-target flags). The project has exactly one deployment shape documented (`spring-qc-app/deploy/README.md`): a single on-site "edge box," with Docker Compose as the only orchestration.

---

8. Setup, Run, Test, Deploy

All commands below are copied/verified against `spring-qc-app/README.md` and the actual `Dockerfile` / `docker-compose.yml` / `.bat` files — not assumed.

Prerequisites

- Docker Desktop (Windows/Mac) or Docker Engine + Compose (Linux) — **recommended path**, or
- Python 3.10+ and a reachable PostgreSQL — for the no-Docker path.
- An NVIDIA GPU + driver + NVIDIA Container Toolkit for GPU inference (optional — CPU fallback documented, but RTSP won't keep up on CPU with more than one concurrent job per the README).

Install & run — Docker (recommended)

```
cd spring-qc-app
cp .env.example .env          # then edit .env and set a real POSTGRES_PASSWORD
docker compose up -d --build
```

Starts `db` (Postgres 16, `localhost:5432`) and `app` (`localhost:8001`). Tables are created automatically on first start (`init_db()`).

```
docker compose logs -f app      # follow logs
docker compose down             # stop (keeps data)
docker compose down -v          # stop and wipe the database + volumes
```

CPU-only build:

```
TORCH_INDEX=https://download.pytorch.org/whl/cpu docker compose build
# and comment out the `deploy:` block under `app` in docker-compose.yml
docker compose up -d
```

Install & run — without Docker

```
cd spring-qc-app
python -m venv .venv
.venv\Scripts\activate        # Windows; Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt
set DATABASE_URL=postgresql://spring_qc:<password>@localhost:5432/spring_qc
python -m uvicorn app:app --host 0.0.0.0 --port 8001
```

On Windows, `setup.bat` then `run.bat` automate the same — **but you still need a reachable Postgres and `DATABASE_URL` set yourself**; `run.bat` does not start one. The easiest source is `docker compose up -d db` even if you otherwise run the app outside Docker.

Tests

None exist in the repository — no `tests/` directory, no `pytest` / `unittest` files found anywhere, and no test runner is configured. This is a real gap — see §9.

Build

The "build" artifact for the actual product is the Docker image (`docker compose build` / `--build`). Separately, `spring-qc-exe/` builds native Windows `.exe` packaging around the same app (`server/build.bat` , `desktop/build.bat` , `hmi/build.bat` , each driving PyInstaller) — this is packaging, not an alternate implementation.

CI/CD

None found. No `.github/workflows/` , no `.gitlab-ci.yml` , no Jenkins/ CircleCI/other pipeline config anywhere in the repo. All builds, tests (if any were run), and deploys are manual.

Deploy (production / on-site)

Fully documented in `spring-qc-app/deploy/README.md` — summarized:

1. One "edge box" PC (GPU, static LAN IP) runs `docker compose up -d --build` .
2. Windows: run `deploy/open-firewall.ps1` (elevated) once to allow inbound 8001 — deliberately **not** 5432.
3. Admin PC: browser → `http://<edge-ip>:8001/admin` . No install.
4. Operator touch panels: kiosk-mode Chrome/Chromium pointed at `http://<edge-ip>:8001/operator` , launched via `deploy/kiosk-operator.bat` (Windows, via `shell:startup`) or `deploy/kiosk-operator.sh` + `deploy/spring-qc-kiosk.service` (Linux systemd `--user` unit).

---

9. Known Issues and Technical Debt

- **No authentication anywhere in the HTTP API** (§3/§6). Explicitly acknowledged in `spring-qc-app/deploy/README.md` , mitigated only by network isolation, not application logic. A real risk if this is ever deployed somewhere less controlled than a single isolated shop-floor LAN.
- **Operator "sign-in" has no server-side session** — `/operators/verify` checks a PIN once; the UI then just remembers the operator's name in browser `localStorage` (`static/operator.html`). Any other API call made from that browser is not actually authorized as that operator.
- **Weak default DB credentials**: `app.py` 's `DATABASE_URL` default and `docker-compose.yml` 's `POSTGRES_PASSWORD` default both fall back to the literal string `spring_qc` if not overridden. The README tells the operator to change it, but nothing enforces that.
- **No foreign-key constraints** between `operators.name` and the `operator` columns in `inspections` / `assignments` (§5) — deleting or renaming an operator leaves `inspections` history pointing at a name that no longer exists in

`operators`. Low risk in practice (delete is manual, rename isn't exposed in the API) but worth knowing before touching that code path.

- **No automated tests of any kind** (§8) — any change to `detector.py`'s tracking/confirmation logic or `app.py`'s API is currently verified only by manual testing.
- **No dependency lockfile** — `requirements.txt` uses floor-only version pins (`>=`), so a fresh install today can resolve different exact versions than one from months ago. Combined with no tests, this is a real "it worked yesterday" risk.
- **Leftover SQLite artifact**: `spring-qc-app/qc.db` (a real, ~20 KB SQLite file) sits in the repo despite `.gitignore` excluding it — almost certainly a stray dev-time file rather than something intentionally shipped; `migrate_from_sqlite.py` is the only code that would ever read it, and only when explicitly pointed at it.
- **`spring-qc-exe/` has ~9 leftover build-session log files** directly in its root (`regen_*.log`, `rebuild_*.log`, `desktop_build.log`) — these are debug output from interactively driving PyInstaller builds, not part of the actual packaging source, and are safe to delete.
- **PIN pad is not rate-limited** (noted directly in `deploy/README.md`) — acceptable per that doc's own stated threat model (trusted shop floor), not acceptable if the deployment model ever changes.
- **No TODO/FIXME/HACK comments found anywhere** in the Python/JS/HTML source — either the code is genuinely clean of known-incomplete markers, or such notes were never left in the first place; can't distinguish which from the repo alone.

---

10. Work in Progress

Not determinable from code. The Git repository at `D:\SPRING` (the root containing both `spring-qc-app/` and `spring-qc-exe/`) currently has **zero commits** — `git log` returns "your current branch 'master' does not have any commits yet," and there are no other local branches. There is no commit history to summarize and no way to tell from the repo what was recently changed, by whom, or why. Everything in this document was derived from the current state of the files on disk, not from history.

---

11. Gotchas

- **Config changes don't affect already-running jobs.** `/save-config` and `/save-roi` rewrite `config.yaml`, which only takes effect on the *next* `/start` call — except per-operator ROI edits specifically, which *do* push live to a running job (`/operators/{name}/roi` calls `det.set_roi` directly). Easy to mix these two up.
- **`started_at` is nullable** on older `inspections` rows (added via an in-place `ALTER TABLE` migration, §5) — any code reading it must handle `None`, as `reports.export.xlsx`'s `clock_pair()` already does. New code touching this table should do the same.
- **S.No is per-operator, not per-job.** A job restart for the same operator continues the sequence from `MAX(s_no)+1`, not from 1 — don't assume S.No uniquely identifies a row within a single job/video.
- **Port 5432 is published by the `db` container on all interfaces**, but is only kept unreachable from the LAN by the host firewall *not* opening it (`deploy/open-firewall.ps1` intentionally skips it). Anyone who changes the firewall config without reading that file's comments could accidentally expose Postgres to the shop floor.
- **Multipart upload size**: FastAPI's normal `UploadFile = File(...)` shortcut would reject anything over Starlette's 1 MiB default part size; `/upload` works around this by calling `request.form(max_part_size=...)` directly. Don't "simplify" this back to the shortcut form without re-reading the comment in `app.py` explaining why.
- **Temp directory redirection**: `app.py` points Python's `tempfile` module at a local `_tmp/` folder instead of the OS default, specifically so large video upload spooling doesn't fill up the system drive. Any new code that uses `tempfile` elsewhere in the process inherits this redirection automatically (it's a module-level `tempfile.tempdir` assignment) — don't be surprised to find temp files in a non-standard location while debugging.
- **The GPU build is the default**, not an opt-in — `docker-compose.yml` and the `Dockerfile` assume CUDA 12.4 unless you explicitly override `TORCH_INDEX` and comment out the `deploy:` GPU reservation block. A plain `docker`

`compose up -d --build` on a non-GPU host will not fail loudly in an obvious way tied to this — read the README's CPU section first.

- `spring-qc-exe/` **is a separate, independent packaging concern.** If you're only working on the actual application logic, you never need to touch it; it only matters when producing a native Windows installer.

---

12. Handover Checklist

> Placeholders below — to be filled in by the outgoing/incoming owner, not determinable from the repository itself.

- **Owner / primary contact**: _____
- **Secondary contact**: _____
- **Credentials location** (vault/password-manager name only — not the secrets themselves): _____
- **Access needed** (repo, deployment host, camera/RTSP credentials, any accounts): _____
- **Pending tasks**: _____
- **Upcoming deadlines**: _____
- **Physical/site access** (edge box location, touch panel locations):

Confidence and Gaps

- **High confidence**: tech stack, API surface, data model, feature descriptions, and the auth/security posture — all read directly from `app.py`, `detector.py`, `docker-compose.yml`, and the existing `README.md` / `deploy/README.md`, which were already unusually thorough and consistent with the code.
- **Could not determine**: any staging/production environment split (none exists), CI/CD (none exists), test coverage (no tests exist), and any project history or in-progress work (zero git commits at the repo root). These aren't guesses — the absence itself is the finding.
- **Not fully verified**: exact installed dependency versions (only floor pins are declared; the actual resolved versions depend on when `pip install` was last run and weren't re-derived from a lockfile, because none exists). The Python runtime version has two different stated minimums (3.10+ in the README, 3.12-slim in the Dockerfile) — both are accurate for their respective contexts but worth knowing they differ.
- **Scope note**: this document focuses on `spring-qc-app/` as "the project" per the task brief's intent (features, API, data model, etc. all only make sense for the actual running application). `spring-qc-exe/` is described structurally in §1 and §8/§11 but not exhaustively, since it's a packaging wrapper with no business logic, API, or data model of its own.